OBJECT ORIENTED PROGRAMMING

CSC166 — Semester II

UNIT 1

Introduction to Object Oriented Programming

Lecture Notes by: Dabbal Singh Mahara

Amrit Campus, Thamel, Kathmandu

Reference: Robert Lafore — OOP in C++ (4th Ed.) | Herbert Schildt — C++ The Complete Reference (4th Ed.)

1. Overview of Structured Programming

1.1 What is Structured Programming?

Structured programming (SP) is a programming paradigm that emerged in the late 1960s and 1970s as a solution to the infamous "software crisis" — the difficulty of writing large, correct, and maintainable programs using unstructured GOTO-based code. It enforces a clear, logical structure through three fundamental control constructs:

- **Sequence:** Instructions execute one after another in a top-down order.
- **Selection:** Decision-making using if, if-else, and switch statements.
- **Repetition / Iteration:** Loops such as for, while, and do-while.

Languages that embody this paradigm include C, Pascal, FORTRAN (modern), and COBOL. In structured programming, a large problem is broken down into smaller sub-problems, each solved by a function or procedure — a technique called procedural decomposition.

1.2 Core Concepts of Structured Programming

a) Functions / Procedures

A function is a named, reusable block of code that performs a specific task. Programs are built as a collection of functions that call each other. The entry point is typically the main() function.

b) Top-Down Design

Programmers start with the overall problem at the top and progressively decompose it into smaller, manageable functions — a technique called stepwise refinement.

c) Global vs. Local Data

Data can be global (accessible by all functions) or local (accessible only within the function that declares it). Structured programs rely heavily on passing data between functions via parameters.

1.3 Limitations of Structured Programming

While structured programming was a significant improvement over unstructured code, it showed serious limitations as programs grew in size and complexity:

- **Data and Functions are Separate:** Data (variables) and the functions that operate on them are not grouped together. Global data can be accessed and modified by any function, leading to unintended side effects.
- **Poor Modeling of Real World:** The real world consists of objects (a car, a bank account, a student). Structured programming models the world as actions (functions), not as things. This creates a mismatch.
- **Difficulty in Reuse:** Functions written for one program are often hard to reuse in another because they depend on global data structures.
- **Maintenance Challenges:** In large systems, a change in a data structure can require changes in many functions across the codebase. This tight coupling makes maintenance costly.
- **Scalability Issues:** As projects grow (tens of thousands of lines), structured programs become difficult to understand, debug, and extend.

> **Note:** The central problem: In structured programming, data is exposed and unprotected. Any function can accidentally (or intentionally) corrupt shared data. This led to the need for a new paradigm — Object Oriented Programming.

2. Object Oriented Programming Approach

2.1 What is Object Oriented Programming?

Object Oriented Programming (OOP) is a programming paradigm that organizes software design around data, or objects, rather than functions and logic. An object is a self-contained unit that bundles both data (attributes) and the functions (methods) that operate on that data.

OOP was formalized by Alan Kay in the Smalltalk language (1970s). Today, it is implemented in languages such as C++, Java, Python, C#, and Ruby. In this course, we use C++, which is a hybrid language supporting both structured and object oriented programming.

> **Note:** Key Insight (Lafore, Ch.1): OOP views a program as a collection of interacting objects, just as the real world is composed of interacting entities. The programmer's job is to model real-world objects in software.

2.2 The Building Block: Object

An object is a software entity that combines:

- **Data (Attributes / Member Variables):** Represent the state or properties of the object. E.g., a BankAccount object has a balance, accountNumber, and ownerName.
- **Functions (Methods / Member Functions):** Represent the behavior or capabilities of the object. E.g., a BankAccount object can deposit(), withdraw(), and getBalance().

This bundling is the essence of OOP. The data and the functions that act on it are always together.

2.3 The Blueprint: Class

Before creating objects, the programmer defines a class. A class is a template, blueprint, or user-defined data type that specifies:

- What data members (attributes) each object of this class will have.
- What member functions (methods) each object of this class can perform.

The relationship between a class and an object is the same as between a blueprint and a house. Many houses (objects) can be built from the same blueprint (class). In C++:

```
class BankAccount { ... };      // Class definition (blueprint)
BankAccount acc1, acc2;         // Objects (instances)
```

2.4 How OOP Overcomes SP Limitations

Feature	Structured Programming	OOP
Focus	Actions (functions)	Objects (data + behavior)
Data Access	Global; exposed to all functions	Encapsulated; hidden inside objects
Real-world Modeling	Poor; world modeled as tasks	Natural; world modeled as entities
Reusability	Difficult; tied to global data	High; classes are reusable units
Scalability	Hard to manage in large projects	Modular; easier to manage and extend
Maintenance	Change in data affects many functions	Change is localized to the class

3. Characteristics of Object-Oriented Languages

OOP languages are identified by four fundamental characteristics (pillars). These are not mere features — they are the conceptual foundation that gives OOP its power. Schildt (C++ Complete Reference) and Lafore both treat these as essential.

3.1 Encapsulation

Encapsulation is the mechanism of wrapping data and the functions that operate on that data together as a single unit (class) and restricting direct access to the internal data from outside the class.

Why Encapsulation?

- Protects data from accidental or unauthorized modification (data hiding).
- Simplifies the interface — users of an object only need to know what methods to call, not how the data is stored.
- Reduces complexity and increases security of the program.

How it works in C++:

Class members are declared with access specifiers:

- **private:** Members accessible only within the class. This is data hiding.
- **public:** Members accessible from anywhere. Typically, member functions are public.
- **protected:** Members accessible within the class and its derived classes (used in inheritance).

> **Note:** Analogy: A car engine is encapsulated under the hood. The driver uses the steering wheel, pedals, and gear shift (public interface) without knowing the internal mechanics (private data). This separation is encapsulation.

3.2 Abstraction

Abstraction is the concept of hiding complex implementation details and showing only the essential features of an object. It means representing only relevant information to the outside world.

- Abstraction defines what an object does, while encapsulation defines how it does it.
- A class itself is an abstraction — it captures the essential properties and behaviors of a real-world entity while ignoring irrelevant details.

- In C++, abstraction is achieved through classes, access specifiers, and (for pure abstraction) abstract classes with pure virtual functions.

Levels of Abstraction:

- **High-level abstraction:** The user of a BankAccount object calls withdraw(500) and does not know how the balance is updated internally.
- **Low-level abstraction:** The programmer of the BankAccount class knows all internal details.

> **Note:** Analogy: When you use a TV remote, you press the volume button without knowing the electronics inside. The remote provides an abstraction of the TV's internal circuitry.

3.3 Inheritance

Inheritance is the mechanism by which a new class (derived class or child class) acquires the properties and behaviors of an existing class (base class or parent class). It represents an IS-A relationship between classes.

- Promotes code reusability — a derived class inherits all public and protected members of the base class.
- Allows a programmer to extend or specialize existing classes without modifying them.
- Supports hierarchical classification of objects.

Example:

```
class Animal { void eat(); void breathe(); };
class Dog : public Animal { void bark(); };   // Dog IS-A Animal
```

Dog inherits eat() and breathe() from Animal, and adds its own bark() method. This avoids rewriting common behavior.

Types of Inheritance (Preview — covered in detail in Unit 5):

- Single Inheritance: One derived class, one base class.
- Multiple Inheritance: One derived class, multiple base classes.
- Multilevel Inheritance: A chain of inheritance (A → B → C).
- Hierarchical Inheritance: Multiple derived classes from one base class.
- Hybrid Inheritance: A combination of the above types.

3.4 Polymorphism

Polymorphism means "many forms." It is the ability of an object, function, or operator to take on multiple forms depending on the context. In OOP, it allows the same interface to be used for different underlying data types or implementations.

Two Types of Polymorphism:

- **Compile-time Polymorphism (Static Binding / Early Binding):** Resolved at compile time. Achieved through function overloading and operator overloading. The compiler decides which function to call based on the function signature.
- **Run-time Polymorphism (Dynamic Binding / Late Binding):** Resolved at runtime. Achieved through virtual functions and pointers/references to base class objects. The correct function is chosen while the program is running.

Example of Function Overloading (Compile-time Polymorphism):

```
void print(int x);      // version 1
void print(double x);   // version 2
void print(string x);   // version 3
```

The function print() takes on different forms depending on the argument type.

> **Note:** The four pillars — Encapsulation, Abstraction, Inheritance, Polymorphism — work together. Encapsulation protects data. Abstraction simplifies interfaces. Inheritance promotes reuse. Polymorphism enables flexible and extensible designs.

4. Additional OOP Concepts

4.1 Message Passing

In OOP, objects interact with each other by sending messages. A message is simply a call to a member function of an object. When Object A calls a method on Object B, it is said to be "passing a message" to B. Object B responds by executing the appropriate method.

```
acc1.deposit(5000);    // Message: 'deposit 5000' sent to acc1
```

4.2 Dynamic Binding

Dynamic binding (also called late binding) means that the code associated with a function call is not determined at compile time but at runtime. This is essential for polymorphism and is implemented via virtual functions in C++. The opposite, static binding, is determined at compile time.

4.3 Extensibility

OOP programs are naturally extensible. New classes can be added with minimal changes to existing code. The base classes remain unchanged; new derived classes are added to extend functionality. This is the principle behind the Open/Closed Principle — software entities should be open for extension but closed for modification.

5. Key Terminology — Quick Reference

Class	A user-defined blueprint/template that defines data members and member functions. It is not itself an object.
Object	An instance of a class. A concrete entity with its own copy of the class's data members.
Attribute	A data member (variable) defined inside a class that represents the state of an object.
Method	A member function defined inside a class that represents the behavior of an object.
Encapsulation	Bundling data and methods together and restricting access to internal data.
Abstraction	Exposing only essential features and hiding implementation complexity.
Inheritance	Deriving a new class from an existing class, reusing and extending its properties.
Polymorphism	The ability of a function/operator/object to behave differently in different contexts.
Message Passing	The mechanism of interaction between objects via method calls.
Dynamic Binding	Resolving a function call at runtime rather than compile time.

6. Unit Summary

This unit established the conceptual foundation for the entire course. The key takeaways are:

- Structured programming uses functions and top-down design but fails to protect data and model the real world naturally.
- Object Oriented Programming addresses these limitations by organizing programs around objects — self-contained units of data and behavior.
- A class is the blueprint; an object is the instance created from that blueprint.
- The four pillars of OOP are: Encapsulation (data protection), Abstraction (simplification), Inheritance (reusability), and Polymorphism (flexibility).
- C++ is a hybrid language supporting both structured and object oriented programming, making it ideal for learning OOP concepts from a structured programming background.

> **Note:** Exam Tip: Be prepared to compare structured programming vs. OOP (differences table), define and give an example of each of the four pillars, and explain why OOP is preferred for large-scale software development.

7. Review Questions

1. Define structured programming. What are its three fundamental control constructs?
2. What is the "software crisis" and how did OOP emerge as a solution?
3. Differentiate between a Class and an Object with an example.
4. Explain Encapsulation and Abstraction. How are they different from each other?
5. What is Inheritance? Give a real-world analogy and a code example.
6. Explain the two types of Polymorphism with examples.
7. List and explain five limitations of structured programming that OOP overcomes.
8. What is dynamic binding? How does it differ from static binding?
9. Draw and explain the comparison between structured programming and OOP using a table.
10. Why is C++ called a hybrid language? What advantages does this offer?